

MLB Scion — Training Run Strategy

Long single run vs. batching, and how to select the ensemble honestly

Companion to lgbmBinaryCLASSIFIER V26.07 · 25 June 2026

1. Bottom line

Two things were tangled together in the question — *how often the run finishes and ranks* (a cadence problem) and *how you choose which models make the ensemble* (a selection problem). The cadence problem is now largely solved in code without batching; the selection problem is the one that actually matters for ROI, and it needs changing regardless of cadence.

- **Cadence.** You no longer need to batch just to see results. The log now writes **one row per model as it completes** and each epoch is logged live to MLflow, so a 1,000-model run is monitorable from hour one and is crash-safe. Keep the single long run.
- **Selection.** Stop selecting models on the outsample — both your current “precision and recall > 0.5 on the outsample” rule and the downstream “rank by realised ROI” are selection on the test set, the same bias we fixed in the trainer. Select on the new `include_in_ensemble` flag (validation-based), and use outsample ROI only as a report-only confirmation.
- **If you still want to batch** (for hard checkpoints), it is methodologically safe but operationally hazardous as-is — same-day batches overwrite each other’s models and log. Use the offset/subfolder convention in §5 if you go that way.

2. What changed in the code (V26.07)

Two additions land directly on your question:

1. `include_in_ensemble` column. Each model is flagged `1` when its **validation** AUC clears a floor (default 0.5, set with `-F`). This is the operational form of “ensemble every model above a sanity floor rather than cherry-pick a top-X” — and crucially the floor is on the honest insample holdout, not the outsample. The end-of-run summary prints how many cleared it.
2. **Incremental, crash-safe log.** The log CSV is appended row-by-row as models finish (header once), then overwritten with the sorted version at the end. A run that dies at epoch 900 keeps 1–899. You can `tail` or open the log mid-run and already see `val_auc_best` and `include_in_ensemble` for every finished model.

Together these mean the thing batching was really for — *getting actionable rankings before the run ends* — now happens continuously inside one run.

3. Why your current mid-training selection is the real risk

The cadence is harmless. The selection rule is not. Copying partial trees and keeping the ones whose **precision and recall both exceed 0.5 on the outsample** is selecting models on the very data you then use to judge the ensemble — identical in kind to the test-AUC ranking we removed from the trainer, and it inflates expected live performance the same way.

The downstream pipeline has the same trap. `batch_analysis.py` ranks models by **realised ROI on the outsample**, and the README frames a weak AUC-vs-ROI correlation as justification for ROI-based selection. But weak correlation only means AUC doesn't predict ROI — it does **not** mean outsample ROI predicts **future** ROI. Both are noisy measurements on the same period; selecting on outsample ROI just overfits the evaluation set in dollars instead of AUC.

Two more reasons to drop the precision/recall-at-0.5 gate specifically:

- **It is threshold-sensitive.** Precision and recall at the default 0.5 cut move around with class balance and the arbitrary cut point. Validation **AUC** is threshold-free and a cleaner gate — which is what `include_in_ensemble` uses.
- **It is a tiny, noisy sample.** “Both > 0.5, which is not often” means you are keeping the handful that happened to look good on one outsample — the textbook way to assemble a backtest that doesn't replicate live.

4. Recommended workflow

Keep one long run; select on validation; confirm on outsample.

1. **Run the full 1,000 in one go.** With live logging + MLflow there is no waiting penalty and no batching hazards. Let it run.
2. **Monitor live.** `tail -f` the log (or sort the MLflow runs by `val_auc`). The `include_in_ensemble` column tells you in real time which finished models are candidates.
3. **Form the prototype ensemble from `include_in_ensemble = 1`** — i.e. every model above the validation floor, not a hand-picked few. Given that validation barely predicts test (corr $\approx$ 0.19), breadth and variance-reduction are where the benefit is; aggressive selection mostly fits noise.
4. **Then run `dTREEPREDS` / `sports_roi` as a report-only check**, not a selector. Look at outsample ROI and the surprise-free subset to **understand** the ensemble, not to choose its members.
5. **Keep a final untouched holdout** (or use genuinely forward live games) for the chosen ensemble's honest number. Nothing that selected the models should appear in it.

Net effect on your routine: you still inspect models mid-run exactly as you like — but the keep/drop decision moves from “precision & recall > 0.5 on the outsample” to “`include_in_ensemble = 1`”, and the outsample becomes confirmation rather than the gate. Same habit, honest basis.

5. If you do batch — the two hazards and the fix

Batching is methodologically fine: `random.seed()` is called with no argument, so every process draws fresh entropy and separate batches produce genuinely different models (no duplication). But two operational hazards bite as the script stands:

Hazard	Why
Same-day overwrite	The model folder is named by DATE only (..._YYYYMMDD) and models are numbered 1..N per run. Two batches on the same day write to the same folder and the second overwrites the first's .pkl files and the log.
Number collisions	Each batch numbers its models 1..200, so model_7 from batch 1 and batch 2 are different models with the same name — merging the logs/pkls needs renumbering.
No global ranking	Each 200-batch ranks only its own 200; you get five local rankings, not one global one, unless you pool and re-rank.

Workable batching convention, if you want checkpoints:

- Give each batch its **own dated/numbered subfolder** (e.g. ..._run01/, ..._run02/) so nothing overwrites.
- Apply a **model-number offset** per batch (batch 2 → 201–400, etc.) when you copy pkls into the shared ensemble pool, so names stay unique.
- **Pool the per-batch logs and re-rank by `val\_auc\_best`** across the union. Because the validation split is the deterministic time-ordered tail of the same insample, **val_auc_best** is directly comparable across batches — so a pooled ranking is valid.

That said, given live logging now exists, the only thing batching buys you over one long run is a hard checkpoint every few days. If that is the goal, prefer larger chunks (say 250–500) with the subfolder + offset convention — not many small 200s.

6. Runtime sense-check and efficiency actions

A follow-up question: is 1–2 weeks for ~1,000 models **reasonable**, or a sign the script is inefficient? Verdict: **reasonable for the work being requested — not a sign anything is broken — but the defaults were doing more expensive work per model than this low-signal problem needs.**

The arithmetic: 1–2 weeks ≈ **10–20 minutes per model**. Each model is a 100-iteration search with 3–10-fold CV, so a few hundred tree-ensemble fits per epoch × 1,000 epochs — hundreds of thousands of fits. That lands squarely at 1–2 weeks, so the timing is consistent with the configured work, not with a performance bug.

Three efficiency changes were made (still V26.07 — no version bump):

- **DART now off by default** (**allow_dart**, flag **-D**). DART was ~⅓ of the booster candidates and is typically 2–5× slower than gbdx for marginal gain on low-signal tabular data. The search now defaults to **gbdx + rf**; dropping DART alone can roughly halve runtime. It remains available (**-D 1**) for problems that benefit — the script is a general classifier, so flexibility is retained.
- **`n\_estimators` search capped at 500** (**max_estimators**, flag **-E**, was 1000). The best models on a near-coin-flip signal don't need 1,000 trees, and early stopping protects the upper end. Configurable for problems that genuinely need larger ensembles.

- ``njobs=-1`` typo corrected to ``n_jobs=-1``. It was silently ignored before (LightGBM defaulted to all cores anyway), so behaviour is unchanged — but the thread setting is now explicit and correct.

Two further optional levers, unchanged in default but worth knowing: set `enable_shap=0` for the bulk search (per-epoch SHAP + plot rendering for 1,000 models is pure overhead — compute SHAP only on the final ensemble), and lower `-f` (searches per fold). Given that validation barely predicts test (corr $\approx$ 0.19), these buy **breadth** faster, which is worth more here than an exhaustive search per model.

Diagnostic, now trivial thanks to the incremental log: after the first handful of epochs, read the `time_taken` column and multiply by your epoch count. ~ 12 min/epoch $\Rightarrow \sim 8$ days is exactly right; ~ 2 min/epoch $\Rightarrow$ under two days, and something else would explain any larger gap.

7. Empirical confirmation: correlation analysis across two 1,000-model runs

Two completed 1,000-model runs — **V123** (stats-only) and **V119** (line + stats) — were analysed to test whether insample model-quality metrics predict out-of-sample performance, and how much apparent edge comes from selecting on the outsample. Both point the same way; V119 more starkly.

Metric	V123 (stats-only)	V119 (line+stats)
Models	1,000	1,000
Mean test AUC (OOS)	0.520	0.528
corr(CV score, test AUC)	+0.19	+0.09
corr(train AUC, test AUC)	~ 0.00	+0.05
Top-15 by TEST AUC (mean test)	0.535	0.542
Top-15 by CV score (mean test)	0.519	0.527
Pool mean test AUC	0.520	0.528

Selecting the top-15 on the honest CV metric returns $\approx$ the pool average (no gain); selecting on test AUC adds $\sim +0.01$ – 0.015 AUC of phantom edge.

- **Insample metrics barely predict out-of-sample.** corr(CV score, test AUC) is just +0.19 (V123) and +0.09 (V119) — under 4% and 1% of the variance. Train AUC, Youden J and Brier all correlate ~ 0 with test AUC. Which model generalises is, to a first approximation, unpredictable from anything measured on the insample.
- **Selecting on the outsample manufactures a phantom edge.** Top-X by CV returns the pool-average test AUC (no improvement); top-X by **test** AUC adds $\sim +0.01$ – 0.015 AUC — pure selection luck on that one window that will not replicate live.
- **Overfitting is visible in the insample itself** (V119): corr(CV score, train AUC) = **-0.45**. The hardest-fitting models (train AUC up to 1.0) score **worse** in cross-validation.
- **DART earned nothing.** In the V119 run the selected boosters were 61% rf / 29% dart / 10% gbdt, yet out-of-sample AUC was identical across them (dart 0.5273, rf 0.5279, gbdt 0.5256). DART was picked a third of the time and added no OOS value while costing the most compute — direct support for making it opt-in.

Implication: ensemble broadly above a validation floor rather than cherry-picking a top-X; treat outsample ROI / precision-recall as confirmation, not selection; and discount ~0.01–0.015 AUC of any top-X-on-outsample backtest “edge” as noise. Both runs are from the V26.06 trainer (DART in the search, sort-by-test selection) — the “before” picture that motivates the V26.07 changes.

8. Feature set sufficiency: evidence from the insample feature analysis

A dedicated association analysis of the full insample (2004–5 Jun 2025, ±150) scored every candidate feature by mutual information (MI) and effect size — ~1,179 features with the bookie line, ~1,170 stats-only — and was cross-referenced against the V119 (38 features) and V123 (35 features) production sets. It answers directly whether the **feature set**, rather than the modelling, is the binding constraint. It is not.

Finding 1 — the signal ceiling is intrinsic, and low

	Line universe (incl. bookie)	Stats-only universe
Candidate features	1,179	1,170
Strongest single feature	G_Bookie_H_MoneyLine	MenOnBase_Strength_YTD
Best-feature MI (share of target entropy)	0.0061 nats (0.88%)	0.0021 nats (0.30%)
Best-feature point-biserial r	~0.098	~0.058
Top-10 cumulative MI (if independent)	4.6% of entropy	2.0%
Cohen's d (every feature)	“negligible”	“negligible”

The single most informative variable in the entire pool — the bookie line itself — explains **under 1% of the outcome's uncertainty**; the best engineered stat, under a third of a percent. Every feature's effect size is graded “negligible.” No feature-selection method can extract signal the data does not contain. This is the efficient-market ceiling, and it matches the model logs (test AUC ≈ 0.52–0.53) exactly.

Finding 2 — selection quality is good; this is not a selection failure

- V119's chosen features sit at a **median MI rank of 75 / 1,179 (top 6%)**; V123's at **100 / 1,170 (top 8.5%)**. You are already picking from the strong end of the distribution.
- They include the genuine top signals — **MenOnBase_Strength_YTD**, **FIP_YTD**, **K-BB%**, **Run_Pythag**, **ClosingProbabilityLine**. None sit in the severe-VIF top-100 (the multicollinear raw counts like **V_NP_Sum** were correctly excluded), so the sets are not internally redundant.
- The **Ivan_BP_*** features rank 3rd–6th by MI but — being highly line-correlated proxies — are **correctly absent from both sets**. Naive top-MI selection would have grabbed four redundant copies of the line; your selection did not.

So the features are well chosen, with low redundancy and the line correctly handled. Re-ranking the same pool will not move the needle.

Finding 3 — two refinements the analysis does surface (not “more features”)

- **V123 is not actually market-blind.** It contains three closing-line-probability features (**ClosingProbabilityLine**), one of which — *G_VHRatio_ClosingProbabilityLine_HV* — is **shared with V119**. So the V119/V123 “consensus” is less independent than the market-anchored-vs-market-blind framing implies: both partially encode the same closing-line signal, which inflates the apparent diversification of their agreement. For genuinely independent confirmation, drop *ClosingProbabilityLine* from V123 (accepting a weaker but truly market-blind model).
- **V123 carries dead weight.** 8 of its 35 features rank *below the median* by MI ($\approx 760\text{--}1,115 / 1,170$): *BallpenERA_Approx*, *Pythag_Luck_Factor*, *Run_Differential_Per_Game*. These sit at the noise floor; trimming them reduces overfitting variance (which the logs show is real — train AUC reached 1.0) at no expected cost to skill.

Bottom line. The feature set is *sufficient* in that it already captures essentially all of the small edge the market leaves on the table; it is *insufficient* only in that this edge is intrinsically tiny — a property of a ± 150 efficient line, not of your selection. Adding features adds complexity and overfitting, not skill. The productive levers remain (a) features targeting **market inefficiency the closing line misses** (line-movement, staleness, news-timing) rather than more box-score aggregates the market already prices, and (b) **calibration over discrimination** to convert the thin edge into bets — which is also where the fave ≈ 0.54 / dog ≈ 0.50 profitability bar is actually met.

Pruning actions — V123 and V119 (recorded 25 Jun 2026)

V123 — remove 11 of 35 features ($\rightarrow 24$), making it genuinely market-blind:

- **3 closing-line features** (so the model no longer sees the market):
V_ClosingProbabilityLine_HV, *V_ClosingProbabilityLine_YTD_HV*,
G_VHRatio_ClosingProbabilityLine_HV.
- **8 dead-weight features** (below-median MI, at the noise floor): *H_BallpenERA_Approx_YTD*,
G_VHRatio_BallpenERA_Approx_YTD, *H_Pythag_Luck_Factor_YTD*,
V_Pythag_Luck_Factor_YTD, *G_VHRatio_Run_Differential_Per_Game*,
G_VHRatio_TotalDistanceTravelled_3G,
G_VHRatio_StartingPitcher_Strikes_Ratio_5G,
G_VHRatio_StartingPitcher_Score_Ratio.

V119 — keep all line features; light pruning only ($\rightarrow 35$, or $\rightarrow 32$ with the optional redundancy trim):

- **Keep** the three line features — *G_Bookie_H_Probability*,
G_VHRatio_ClosingProbabilityLine_HV, and *G_H_CLL_OPL_Prob_Diff* (the closing-minus-opening **line-movement** signal; modest univariate MI but conditionally valuable). These stay precisely to preserve the market-anchored vs market-blind contrast with the now-line-free V123.
- **Remove 3 dead-weight features:** *G_VHRatio_DaysRest* (MI rank 1172/1179),
G_VHRatio_StartingPitcher_Strikes_Ratio_5G (970),
G_VHRatio_TotalDistanceTravelled_3G (924).

- **Do not cut on linear r alone:** `G_VHRatio_StartingPitcher_StrikesImpact_YTD` has low point-biserial r (0.020) but MI rank 84 — its signal is nonlinear, so keep it. For a tree model, trust the MI ranking over linear correlation.
- **Optional redundancy trim:** offense cluster ($\rho \approx 0.96\text{--}0.99$) — keep `G_VHRatio_Weighted_Offense_Index_YTD` (best MI), drop `G_VHRatio_wOBA_YTD` and `G_VHRatio_OPS_YTD`; run-output pair ($\rho \approx 0.90$) — keep `V_MenOnBase_Strength_YTD`, drop `V_Run_Differential_Per_Game_YTD`. Won't change prediction much, but reduces overfitting variance and de-splits SHAP importance.

V123 needs structural surgery (lose the market features + the noise); V119 needs only light pruning, with its line features deliberately retained.

One line: keep the single long run (now live-logged and crash-safe), build the ensemble from `include_in_ensemble = 1` on the validation holdout, and demote outsample precision/recall and ROI from selectors to confirmations.